1 Índice

1. Descripción del sistema RAG 3
1.1 Objetivo del sistema 3
1.2 Fuentes de información 3
1.3 Arquitectura general del sistema 3
2 Arquitectura del sistema 5
2.1 Pipeline de ingestión 5
2.2 Pipeline de consulta 5
2.3 Esquema del sistema RAG 5
3 Estructura del repositorio 6
3.1 Esquema del árbol de directorios 6
4 Instalación y ejecución del sistema 7
4.1 Requisitos 7
4.2 Clonado del repositorio 7
4.3 Instalación de dependencias 7
4.4 Despliegue de la base de datos vectorial 7
4.5 Configuración del archivo de variables de entorno 7
4.6 Ejecución del pipeline de ingesta de datos 8
4.7 Ejecución de la API 8
4.8 Esquema del pipeline de ingesta de datos 9
5 Selección del modelo de embedding 10
6 Selección y diseño de la base de datos vectorial 11
6.1 Organización de los datos 11
6.2 Estrategia de organización de datos para mejorar la búsqueda 11
6.3 Métrica de similitud 11
6.4 Índice de búsqueda 11
7 Estrategia para mantener la información actualizada 13
7.1 Control de cambios en las fuentes de datos 13
7.2 Control de cambios en los chunks indexados 13

7.3 Proceso y periodicidad de actualización de datos 13
7.4 Esquema del proceso de actualización de datos 14
8 Integración del Modelo de Lenguaje Grande (LLM) 15
8.1 Justificación del modelo usado 15
8.2 Uso del LLM en la generación de respuestas (RAG) 15
8.3 Uso del LLM en el sistema de routing semántico 15
8.4 Esquema de la integración del LLM en el sistema 16
9 Estrategia de procesamiento e indexación de información 17
9.1 Estrategia de generación de chunks 17
9.2 Pipiline de indexación tolerante a fallos 18
10 Evaluación del sistema 19
10.1 Pruebas de consultas sobre vulnerabilidades 19
10.2 Pruebas de consultas sobre técnicas de ataque 20
10.3 Pruebas de consultas sobre análisis de amenazas 20
10.4 Preguntas fuera del contexto de las fuentes de datos 21
11 Limitaciones del sistema 22
11.1 Interpretación del contexto por parte del LLM 22
11.2 Ausencia de eliminación automática de información obsoleta 22
12 Mejoras futuras 23
12.1 Optimización del prompt usado por el LLM 23
12.2 Incorporación de un sistema de reranking 23
12.3 Establecimiento de un umbral mínimo de similitud 23

1. Descripción del sistema RAG

En este proyecto se implementa un sistema RAG (Retrieval Augmented Generation) orientado al sector de la ciberseguridad. El sistema es capaz de proporcionar respuestas precisas sobre distintas fuentes de datos que van cambiando constantemente.

1.1 Objetivo del sistema

El sistema cumple con los apartados definidos en el documento de evaluación:

- **Comprensión de conceptos**: Se aplican de forma práctica los principios de bases de datos vectoriales y arquitectura RAG.
- **Gestión de información dinámica**: El sistema es capaz de actualizar datos sobre amenazas y vulnerabilidades en tiempo real
- **Integración y viabilidad**: El modelo de embedding, la base de datos vectorial y el LLM permiten generar respuestas de alta calidad.

1.2 Fuentes de información

Las fuentes de información usadas en el sistema abarcan distintos niveles de inteligencia de amenazas:

- **CISA Known Exploited Vulnerabilities (KEV) Catalog**: Es un catálogo que publica la Agencia de Ciberseguridad e Infraestructura de Estados Unidos donde va recogiendo vulnerabilidades con evidencia confirmada de explotación activa en el mundo real. Este catálogo se actualiza de forma continua y dinámica.
- **MITRE ATT&CK**: Es un catálogo que recoge las tácticas y técnicas usadas por ciberdelincuentes para realizar ataques. Este catálogo lo gestiona una organización americana sin ánimo de lucro llamada MITRE Corporation y se actualiza dos veces al año (abril y octubre) con actualizaciones importantes. De forma continua se realizan actualizaciones menores para corregir errores, matizar algún dato, etc.
- **ENISA Threat Landscape 2025**: Es un informe anual que publica la Agencia de la Unión Europea para la Ciberseguridad (ENISA) que analiza miles de incidentes para identificar las tendencias de ciberseguridad más críticas en la Unión Europea.

1.3 Arquitectura general del sistema

El sistema RAG está compuesta por los siguientes componentes principales:

- **Pipeline de ingestión de datos**: Procesa las fuentes de información, genera los fragmentos de texto (chunks) y crea los embeddings.
- **Base de datos vectorial**: Los embedding generado se almacenan en una base de datos vectorial (Qdrant) lo que permite hacer búsquedas semánticas.
- **Modelo de embeddings**: Se usa un modelo de embeddings para transformar el texto en vectores numéricos que representan un significado semántico.

- **Modelo de lenguaje LLM**: El LLM usa el contexto recuperado de la base de datos para generar la respuesta final.
- **API de consulta**: Para que los usuarios realicen preguntas al sistema se ha desarrollado una API REST con FastAPI.

2 Arquitectura del sistema

La arquitectura del sistema se divide en dos procesos principales:

- **Pipeline de ingestión (offline)**: Se encarga de la descarga y procesamiento de las fuentes de información, así como de generar los embeddings.
- **Pipeline de consulta (online)**: Se encarga de recuperar la información relevante y generar las respuestas.

2.1 Pipeline de ingestión

Procesa las tres fuentes de información que son CISA KEV, MITRE ATT&CK y ENISA de la siguiente forma:

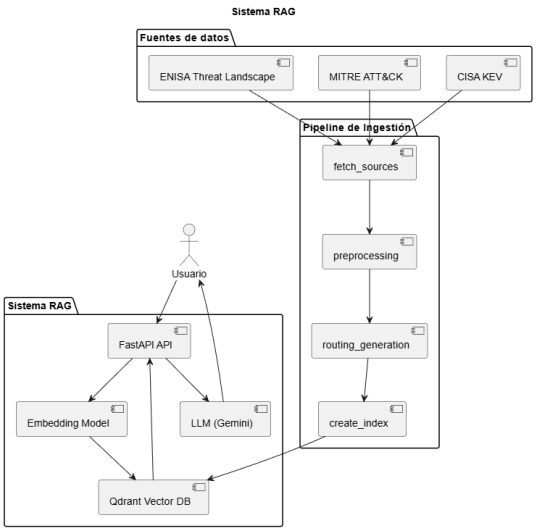
- Se descargan las fuentes de datos.
- Se extrae y se limpia el contenido.
- El texto se divide en fragmentos (chunks).
- Cada fragmento se transforma en un embedding.
- Los embeddings se almacenan en la base de datos vectorial junto con sus metadatos.

2.2 Pipeline de consulta

Cuando el usuario realiza una pregunta el sistema realiza lo siguiente:

- Transforma la consulta en un embedding.
- Se realiza una búsqueda semántica en la base vectorial.
- Se recuperan los fragmentos de información más relevantes.
- Estos fragmentos se usan como contexto para el LLM.
- El LLM genera la respuesta final para el usuario.

2.3 Esquema del sistema RAG



3 Estructura del repositorio

El proyecto tiene una estructura modular que separa los distintos componentes del sistema lo que facilita el mantenimiento y futuras ampliaciones. A continuación, se detalla el árbol de directorios.

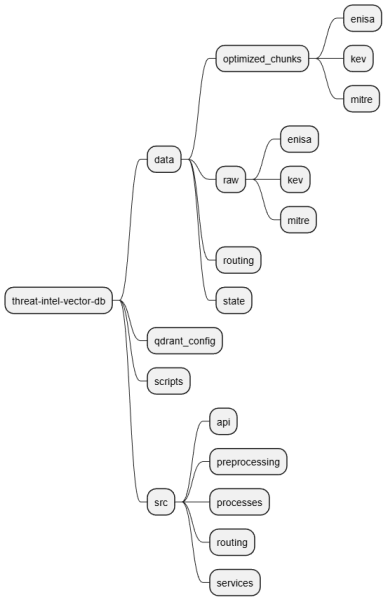
1. **data**: Datos utilizados para el sistema. Este directorio se crea automáticamente al lazar el pipeline de ingesta de datos.
1.1. **optimized_chunks**: Fragmentos de texto generados. Cada fuente tiene un directorio propio.
1.2. **raw**: Fuentes originales descargadas. Cada fuente tiene un directorio que almacena el documento descargado y un archivo json con metadatos.
1.3. **routing**: Resúmenes usados para el routing. Contiene un archivo json por cada fuente y luego otro global.
1.4. **state**: Contine archivos json para el control del estado de la ingesta de datos.

2. **qdran_config**: Contiene la configuración de la base de datos Qdrant.

3. **scripts**: Scripts del pipeline de ingestión de datos.

4. **src**: Contiene los directorios del sistema.
4.1. **api**: API Rest (Fast API)
4.2. **preporcessing**: Scripts para el preprocesado de datos.
4.3. **processes**: Script con la lógica del sistema.
4.4. **routing**: Script con las palabras clave para generar las rutas.
4.5. **services**: Scripts con los servicios externos (base de datos, llm y embeddings)

3.1 Esquema del árbol de directorios



4 Instalación y ejecución del sistema

4.1 Requisitos

Para ejecutar el sistema hay que tener instalados los siguientes programas:

- Python 3.11 o superior.
- Docker Desktop y Docker Compose.
- UV (gestor de dependencias de Python).

Además de las herramientas es necesario tener una API KEY de Google Gemini porque se va a usar para generar embeddings y para el LLM.

La clave se guardará en el archivo .env de la raíz del proyecto.

4.2 Clonado del repositorio

Para clonar el repositorio usamos los siguientes comandos:

- **git clone https://github.com/Gringopel/threat-intel-vector-db.git**
- **cd threat-intel-vector-db**

4.3 Instalación de dependencias

El proyecto usa UV para la gestión de las dependencias y para instalar las librerías necesarias ejecutamos el comando:

- **uv sync**

4.4 Despliegue de la base de datos vectorial

La base de datos vectorial utilizada es Qdrant y se lanza mediante Docker. Por lo tanto, el primer paso es abrir Docker Desktop y en la terminal ejecutar el siguiente comando:

- **docker compose up -d**

Ahora el servicio de Qdrant lo tenemos disponible en la siguiente dirección:

- **http://localhost:6333/dashboard**

4.5 Configuración del archivo de variables de entorno

En el directorio principal hay un archivo de nombre ".env_example" que debemos renombrar a ".env".

Después lo abrimos y ponemos nuestra API KEY en la línea donde pone "GOOGLE_API_KEY=".

Debe quedar así "GOOGLE_API_KEY=AIzaSyAOa……".

4.6 Ejecución del pipeline de ingesta de datos

La ingestión de datos se realiza mediante un script que coordina todos los pasos necesarios.

El pipeline del script tiene los siguientes pasos:

- **fetch**: Descarga las fuentes de datos.
- **preprocess**: Limpia y transforma los datos en fragmentos de texto (chunks).
- **routing**: Genera resúmenes utilizados para el sistema de routing.
- **index**: Genera los embeddings y los almacena en la base de datos vectorial.

Todos estos pasos se pueden ejecutar de una vez con el comando:

- **python .\script\run_pipeline.py --all**

También es posible ejecutar un paso de forma individual con el comando:

- **python .\script\run_pipeline.py --step [nombre del paso]**
  ejemplo "python .\script\run_pipeline.py --step fetch"

También se pueden ejecutar varios pasos al mismo tiempo:

- **python .\script\run_pipeline.py --steps [nombres de los pasos]**
  ejemplo "python .\script\run_pipeline.py --step fetch preprocess"

Esta modularidad permite por ejemplo realizar la descarga de las fuentes sin tener que realizar el resto de pasos.

Una vez finalizado el script en el directorio data/state se generan los siguientes archivos:

- **index_progress.json**: Aquí podemos ver los ids que de los índices que se han procesado.
- **pipeline_report.json**: Archivo donde podemos consultar los scripts (pasos) ejecutados, su estado, tiempo de ejecución , etc.
- **sources_state.json**: En este archivo se guardan metadatos relacionados con las fuentes de datos. Estos datos son por ejemplo la ultima vez que se actualizo, el numero de chunks generados, el estado, la fuente, el hahs para comprobación de cambios, etc.

4.7 Ejecución de la API

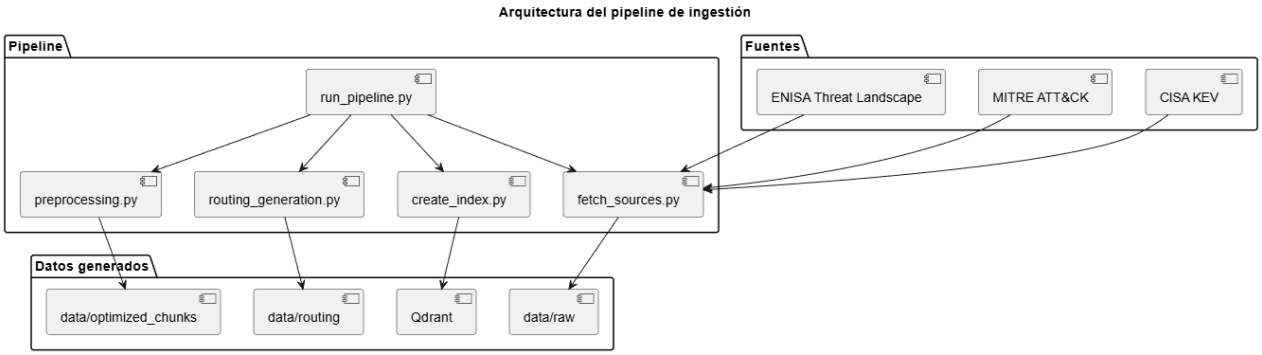
Una vez se ha generado el índice vectorial podemos iniciar la API con el siguiente comando:

- **python -m src.main**

Para empezar a hacer consultas a la API vamos a la siguiente url:

- **http://localhost:8000/docs**

4.8 Esquema del pipeline de ingesta de datos



5 Selección del modelo de embedding

Para este proyecto se ha usado Gemini Embedding de Google y más concretamente el modelo "gemini-embedding-001". A continuación, se detallan los motivos para esta elección:

- Al ser un modelo más antiguo y de solo texto obtenemos un **buen rendimiento** para nuestro objetivo.
- El sistema usa Gemini de Google como LLM lo que permite **mejor integración y menor dependencia** de otros proveedores.
- El principal motivo para esta elección ha sido que el modelo es **multilingüe** ya que las fuentes de datos están en inglés y los usuarios realizan las consultas en español.

No he considerado realizar fine-tuning porque el modelo funciona bastante bien.

6 Selección y diseño de la base de datos vectorial

La base de datos vectorial elegida ha sido Qdrant por los siguientes motivos:

- Es de **código abierto** y permite la búsqueda por similitud.
- **Fácil despliegue** con Docker.
- **Fácil integración** con Python.
- Permite **almacenar metadatos** junto a los vectores.

6.1 Organización de los datos

La información almacenada viene de los chunks generados en el preprocesamiento y cada uno de ellos representa un fragmento semánticamente coherente con el contenido original.

Cada punto en la base de datos contiene un vector embedding del texto, el texto original del fragmento y metadatos estructurados. Estos metadatos estructurados varían dependiendo de la fuente de datos.

Esta forma de estructurarlo permite hacer búsquedas vectoriales combinadas con filtros de metadatos, lo que facilita la recuperación de información relevante y la actualización de datos cuando hay cambios en las fuentes.

6.2 Estrategia de organización de datos para mejorar la búsqueda

Se ha implementado un sistema de routing semántico previo a la búsqueda. En el script de generación de routing se agrupan los chunks por rutas temáticas (routes) y se generan resúmenes representativos con un LLM, lo que permite decidir primero qué fuente o categoría es más relevante, reduciendo el número de vectores que deben compararse y mejorando la precisión de la recuperación.

6.3 Métrica de similitud

Para calcular la métrica de similitud entre vectores se ha usado la similitud del coseno (COSINE) porque en este caso de uso necesitamos identificar textos con un significado similar independientemente de la longitud del texto o la magnitud del vector.

6.4 Índice de búsqueda

El índice de búsqueda que se ha utilizado es el que usa Qdrant por defecto HNSW (Hierarchical Navigable Small World). Este índice permite encontrar vectores semánticamente similares mucho más rápido que otras opciones como puede ser “brute-force”.

Brute-force no es viable para este proyecto porque la base de datos va a ir aumentando el tamaño de vectores a medida que se van actualizando las fuentes de datos, lo que a largo plazo

va a tener costes de tiempo. A nivel de rendimiento tampoco interesa porque se necesitan respuestas en tiempo real.

7 Estrategia para mantener la información actualizada

Debido a que las fuentes de información usadas son dinámicas, el pipeline de ingesta permite la actualización periódica de los datos, añadiendo únicamente aquellos que son nuevos.

Para evitar duplicados y reindexaciones innecesarias el pipeline usa un control de hashes de contenido en dos niveles, uno en las fuentes y otro en cada chunk indexado.

Este sistema aporta tres ventajas principalmente:

- Reduce el coste computacional al procesar solo los datos nuevos.
- Realiza menos llamadas al modelo de embeddings y al llm reduciendo costes.
- Permite tener actualizados grandes volúmenes de datos.

7.1 Control de cambios en las fuentes de datos

En el paso de la descarga de fuentes (fetch) el sistema calcula un hash del contenido descargado y lo almacena en el archivo "*sources_state.json*", junto a otros metadatos de la fuente.

Cuando el pipeline se vuelve a ejecutar y el sistema compara el hash actual con el hash que tiene almacenado. Si el hash ha cambiado se marca esa fuente para se preprocesada en el siguiente paso, por el contrario, si el hash no ha cambiado significa que la fuente no ha tenido modificaciones y se marca para que no sea preprocesada esa fuente.

7.2 Control de cambios en los chunks indexados

Una vez se han marcado fuentes para preprocesar se generan los chunks optimizados, en el paso de la indexación (index) se genera un hash del contenido relevante del chunk, incluyendo el texto y sus metadatos. Este hash queda almacenado en el payload del punto insertado en Qdrant.

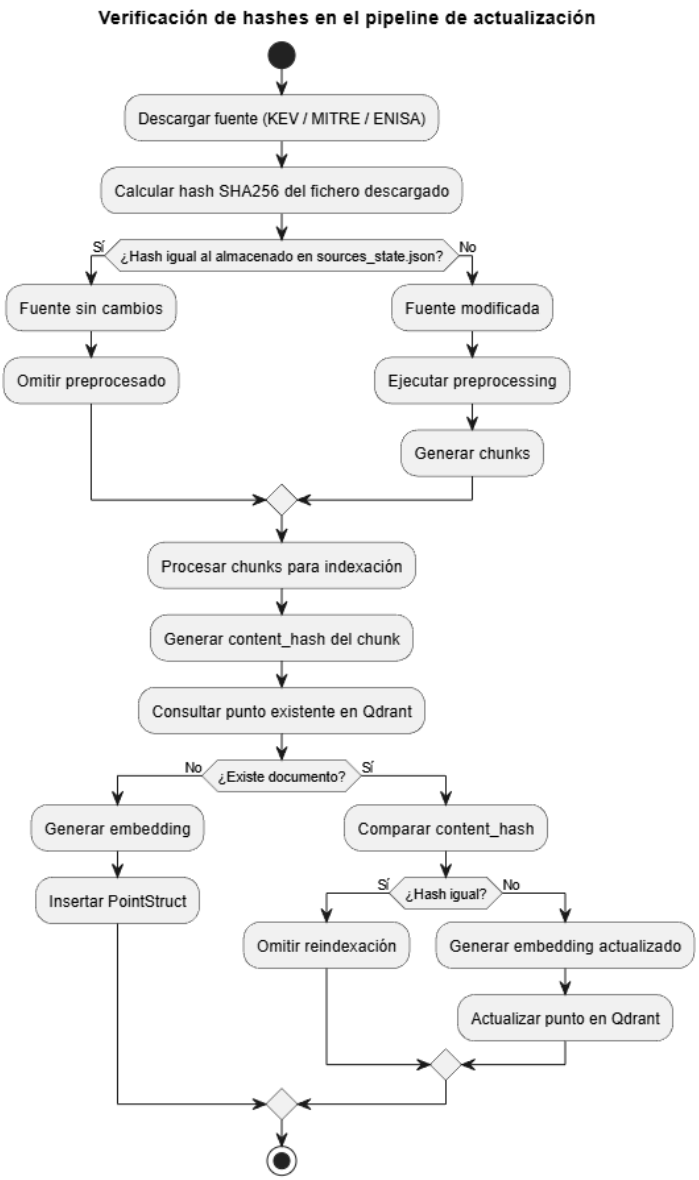
Antes de generar un nuevo embedding o insertar un punto en la base vectorial, el sistema verifica si ese chunk existe en Qdrant comparando los hashes. Si los hashes coinciden el chunk no se indexa en la base de datos.

7.3 Proceso y periodicidad de actualización de datos

Para mantener actualizados los datos se puede ejecutar el pipeline de ingesta de datos completo. También se puede lanzar únicamente el módulo "fetch" de la forma descrita en el apartado 4.6.

Debido a la periodicidad en la que se actualizan las fuentes de datos, descritas en el apartado 1.2, se estima conveniente ejecutar el pipeline de ingesta de datos al menos una vez al día para mantener las fuentes de datos actualizadas.

7.4 Esquema del proceso de actualización de datos



8 Integración del Modelo de Lenguaje Grande (LLM)

8.1 Justificación del modelo usado

En este proyecto se usa el modelo “gemini-2.5-flash-lite” de Google por medio de la librería LangChain. Indicar que se ha configurado una temperatura de 0.2 para que las respuestas sean más deterministas con el contexto recuperado.

La elección de este modelo se debe a que tiene un coste reducido, buen equilibrio entre calidad de generación y la facilidad de integración dentro de lo que ofrece Google. Además este modelo es multilingüe permitiendo interpretar consultas en español y generar respuestas utilizando información recuperada en inglés.

8.2 Uso del LLM en la generación de respuestas (RAG)

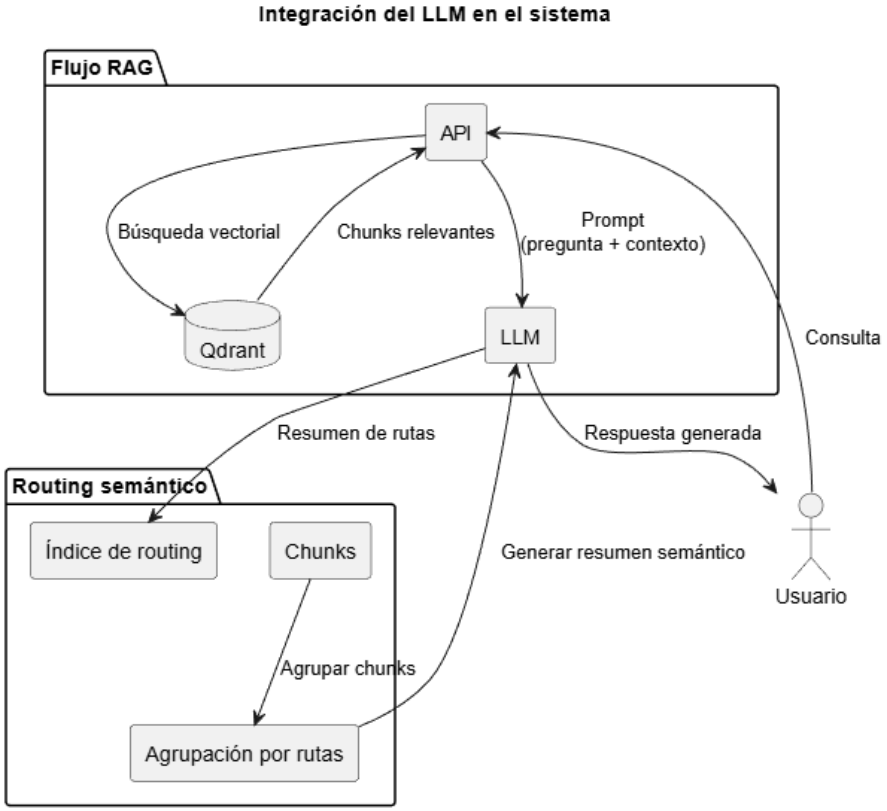
El LLM se usa para generar la respuesta final dentro del RAG. Cuando un usuario hace una pregunta el sistema realiza los siguientes pasos:

- La pregunta se envía al sistema por medio de la API.
- La consulta se transforma en un embedding.
- Se realiza una búsqueda en la base vectorial para recuperar los chunks más relevantes (esta configurado para obtener 5).
- Se construye un prompt combinando la pregunta del usuario y los fragmentos recuperados.
- El LLM genera la respuesta final usando ese contexto.

8.3 Uso del LLM en el sistema de routing semántico

El LLM también se utiliza durante la generación del índice de routing semántico del sistema. En este proceso los chunks de cada fuente se agrupan en rutas temáticas. Para cada grupo se construye un documento representativo (almacenados en la carpeta data/routing) que se envía al LLM para que genere un resumen que describa el contenido de esa ruta. Estos resúmenes se usan después para ayudar al sistema a determinar qué fuentes o rutas son más relevantes.

8.4 Esquema de la integración del LLM en el sistema



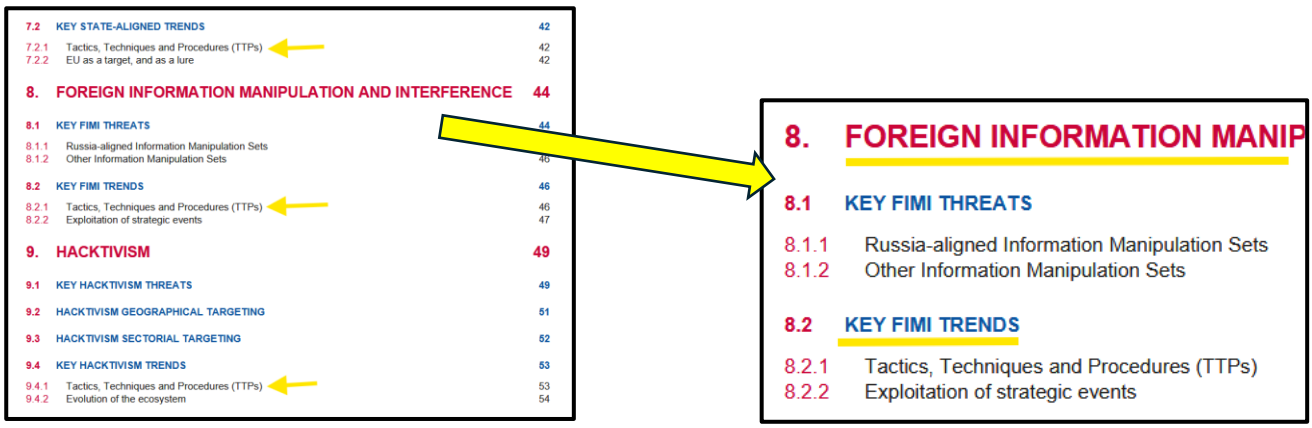
9 Estrategia de procesamiento e indexación de información

9.1 Estrategia de generación de chunks

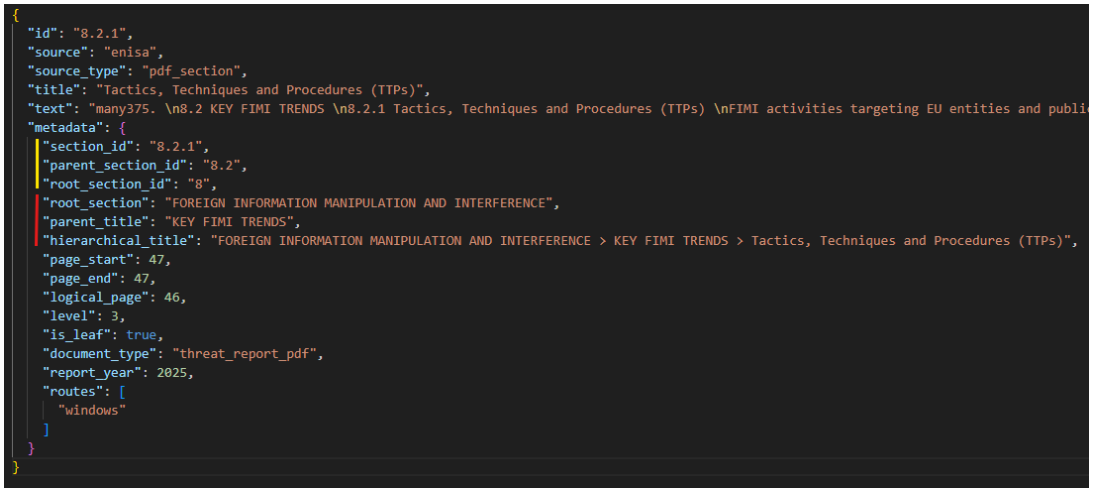
En el proyecto hay fuentes con datos estructurados como son CISA KEV y MITRE ATT&CK y fuentes de datos sin estructurar, por lo que hay que diseñar distintas estrategias para la generación de chunks.

Para los datos estructurados cada chunk representa una unidad lógica de información, como es una vulnerabilidad en la fuente KEV o una técnica de ataque en la fuente MITRE ya que cada fragmento tiene todos los datos relevantes.

La fuente de datos no estructurados (ENISA) es un pdf que no dispone de sumario ni índice por lo que se ha usado el nombre de los apartados para realizar búsquedas dentro del texto y realizar la fragmentación en base a esas secciones/apartados. Un hándicap ha sido que hay secciones con el mismo nombre lo que puede interferir luego en la búsqueda de información. Para resolver este problema se han añadido en los metadatos del chunk datos de las secciones superiores. Como ejemplo de esta problemática se muestra en la siguiente imagen varias secciones que tienen el mismo título.



En esta otra imagen se puede observar los metadatos del chunk donde se guarda esa jerarquía.



```
{
  "id": "8.2.1",
  "source": "enisa",
  "source_type": "pdf_section",
  "title": "Tactics, Techniques and Procedures (TTPs)",
  "text": "many375. \n8.2 KEY FIMI TRENDS \n8.2.1 Tactics, Techniques and Procedures (TTPs) \nFIMI activities targeting EU entities and publi
  "metadata": {
    "section_id": "8.2.1",
    "parent_section_id": "8.2",
    "root_section_id": "8",
    "root_section": "FOREIGN INFORMATION MANIPULATION AND INTERFERENCE",
    "parent_title": "KEY FIMI TRENDS",
    "hierarchical_title": "FOREIGN INFORMATION MANIPULATION AND INTERFERENCE > KEY FIMI TRENDS > Tactics, Techniques and Procedures (TTPs)",
    "page_start": 47,
    "page_end": 47,
    "logical_page": 46,
    "level": 3,
    "is_leaf": true,
    "document_type": "threat_report_pdf",
    "report_year": 2025,
    "routes": [
      "windows"
    ]
  }
}
```

9.2 Pipiline de indexación tolerante a fallos

Se ha identificado un problema durante la implementación del sistema, localizado en el proceso de indexación. Debido al elevado número de chunks (2284) que hay que añadir a la base vectorial, el tiempo para completar este proceso es de casi 30 minutos (el tiempo de ejecución se puede consulta en el archivo "*pipeline_report.json*" en la carpeta data/state).

Ante esta problemática se ha implementado un sistema que permite reanudar el proceso de indexación sin necesidad de reconstruir completamente la base vectorial. Para ello se mantiene un registro de los elementos ya procesados, permitiendo continuar desde el último punto procesado. Este sistema funciona en paralelo al sistema de hashes descrito en el apartado 7.2.

En caso de una interrupción en el proceso de indexación solo habría que ejecutar ese paso con el siguiente comando:

- **python .\script\run_pipeline.py --step index**

10 Evaluación del sistema

La evaluación del sistema se ha realizado mediante pruebas funcionales sobre la API. Estas pruebas permiten verificar el funcionamiento completo del sistema, incluyendo la generación de embeddings, la búsqueda vectorial y la generación de respuestas mediante el modelo de lenguaje.

10.1 Pruebas de consultas sobre vulnerabilidades

Estas pruebas van enfocadas a la fuente KEV. Como ejemplo se realiza la siguiente consulta:

"¿Qué vulnerabilidades de Cisco están siendo explotadas activamente?"

El sistema recuperó varios fragmentos relevantes del catálogo CISA Known Exploited Vulnerabilities (KEV) relacionados con productos Cisco.


```
{
  "question": "¿Qué vulnerabilidades de Cisco están siendo explotadas activamente?",
  "answer": "La información proporcionada no indica si alguna de las vulnerabilidades de Cisco mencionadas está siendo explotada activamente. El campo \"Known ransomware campaign use\" para toda
s las vulnerabilidades listadas es \"Unknown\".",
  "sources": [
    {
      "id": "76ce4898-0a91-5554-b64a-14f3c84adfe4",
      "score": 0.74586755,
      "source": "kev",
      "title": "Cisco IOS and IOS XE Software SNMP Remote Code Execution Vulnerability",
      "text": "CVE: CVE-2017-6742\nVendor: Cisco\nProduct: IOS and IOS XE Software\nVulnerability name: Cisco IOS and IOS XE Software SNMP Remote Code Execution Vulnerability\nDescription: The S
imple Network Management Protocol (SNMP) subsystem of Cisco IOS and IOS XE contains a vulnerability that could allow an authenticated, remote attacker to remotely execute code on an affected sys
tem or cause an affected system to reload.\nRequired action: Apply updates per vendor instructions.\nDue date: 2023-05-10\nKnown ransomware campaign use: Unknown\nCWES: cwe-119\nNotes: https://s
ec.cloudapps.cisco.com/security/center/content/CiscoSecurityAdvisory/cisco-sa-20170629-snmp; https://nvd.nist.gov/vuln/detail/CVE-2017-6742\nSource: CISA Known Exploited Vulnerabilities Catalog"
    },
    {
      "id": "38610462-82a1-5766-af75-c7a3acd4398d",
      "score": 0.74574995,
      "source": "kev",
      "title": "Cisco IOS Software and Cisco IOS XE Software Quality of Service Remote Code Execution Vulnerability",
      "text": "CVE: CVE-2018-0151\nVendor: Cisco\nProduct: IOS and IOS XE Software\nVulnerability name: Cisco IOS Software and Cisco IOS XE Software Quality of Service Remote Code Execution Vuln
erability\nDescription: A vulnerability in the quality of service (QoS) subsystem of Cisco IOS Software and Cisco IOS XE Software could allow an unauthenticated, remote attacker to cause a denia
l of service (DoS) condition or execute arbitrary code with elevated privileges.\nRequired action: Apply updates per vendor instructions.\nDue date: 2022-03-17\nKnown ransomware campaign use: Un
known\nCWES: cwe-119\nNotes: https://nvd.nist.gov/vuln/detail/CVE-2018-0151\nSource: CISA Known Exploited Vulnerabilities Catalog"
    },
    {
      "id": "f0e6c78c-880c-5fd8-8bfc-ae09aeb67bf6",
      "score": 0.7457474,
```


Download

Entre los fragmentos recuperados se encuentran varias vulnerabilidades como son la CVE-2017-6742 y la CVE-2018-0151, e incluye los datos de la vulnerabilidad como es el indicador CVE, el producto afectado, la descripción de la vulnerabilidad, la puntuación de similitud y la fuente de origen.

La respuesta del modelo es:

"La información proporcionada no indica si alguna de las vulnerabilidades de Cisco mencionadas está siendo explotada activamente. El campo \"Known ransomware campaign use\" para todas las vulnerabilidades listadas es \"Unknown\".".

Analizando esta respuesta vemos que la búsqueda vectorial la hace perfectamente, pero la respuesta que genera el LLM no puede determinar si las vulnerabilidades están siendo explotadas activamente porque está teniendo en cuenta el campo *"Known ransomware campaign use"* cuyo valor es *"unknonw"*. Este comportamiento es erróneo ya que el propio catalogo KEV recoge vulnerabilidades que están siendo explotadas activamente, aunque no estén asociadas a campañas de ransomware.

En cambio, si se pregunta:

"¿Qué vulnerabilidades afectan a sistemas macos?"

La respuesta del sistema es la siguiente:


```
{
  "question": "¿Qué vulnerabilidades afectan a sistemas macos?",
  "answer": "Las vulnerabilidades que afectan a sistemas macOS son:\n\n*   **Apple macOS Use-After-Free Vulnerability (CVE-2019-8526):** Una vulnerabilidad de uso después de liberación que podrí
a permitir la escalada de privilegios.\n*   **Apple macOS Out-of-Bounds Write Vulnerability (CVE-2022-22675):** Una vulnerabilidad de escritura fuera de límites en macOS Monterey que podría perm
itir a una aplicación ejecutar código arbitrario con privilegios de kernel.\n*   **Apple macOS Out-of-Bounds Read Vulnerability (CVE-2022-22674):** Una vulnerabilidad de lectura fuera de límites
en macOS Monterey que podría permitir a una aplicación leer memoria del kernel.\n*   **Apple macOS Unspecified Vulnerability (CVE-2021-30713):** Un problema de permisos no especificado en Transp
arency, Consent, and Control (TCC) de Apple macOS que podría permitir a una aplicación maliciosa eludir las preferencias de privacidad.\n*   **Apple macOS Unspecified Vulnerability (CVE-2021-306
57):** Un problema de lógica no especificado en Preferencias del Sistema de Apple macOS que podría permitir a una aplicación maliciosa eludir las comprobaciones de Gatekeeper.",
  "sources": [
    {
      "id": "5af99146-4b2b-50aa-b033-a2862c4a2738",
      "score": 0.74156713,
      "source": "kev",
      "title": "Apple macOS Use-After-Free Vulnerability",
      "text": "CVE: CVE-2019-8526\nVendor: Apple\nProduct: macOS\nVulnerability name: Apple macOS Use-After-Free Vulnerability\nDescription: Apple macOS contains a use-after-free vulnerability t
hat could allow for privilege escalation.\nRequired action: Apply updates per vendor instructions.\nDue date: 2023-05-08\nKnown ransomware campaign use: Unknown\nCWES: cwe-416\nNotes: https://su
pport.apple.com/en-us/HT209600; https://nvd.nist.gov/vuln/detail/CVE-2019-8526\nSource: CISA Known Exploited Vulnerabilities Catalog"
    },
    {
      "id": "ed914977-32f4-5014-8083-17eefad7b4f6",
      "score": 0.7350173,
      "source": "kev",
      "title": "Apple macOS Out-of-Bounds Write Vulnerability",
      "text": "CVE: CVE-2022-22675\nVendor: Apple\nProduct: macOS\nVulnerability name: Apple macOS Out-of-Bounds Write Vulnerability\nDescription: macOS Monterey contains an out-of-bounds write
vulnerability that could allow an application to execute arbitrary code with kernel privileges.\nRequired action: Apply updates per vendor instructions.\nDue date: 2022-04-25\nKnown ransomware c
ampaign use: Unknown\nCWES: cwe-20, cwe-125\nNotes: https://nvd.nist.gov/vuln/detail/CVE-2022-22675\nSource: CISA Known Exploited Vulnerabilities Catalog"
    },
    {
```


Download

Aquí comprobamos que el LLM si interpreta bien los contextos recuperados.

10.2 Pruebas de consultas sobre técnicas de ataque

Estas pruebas van enfocadas a la fuente MITRE ATT&CK. Como ejemplo se realiza la siguiente consulta:

> "¿Qué técnicas utilizan los atacantes para mantener persistencia en un sistema?

La respuesta del sistema es la siguiente:


```
{
  "question": "¿Qué técnicas utilizan los atacantes para mantener persistencia en un sistema?",
  "answer": "Los atacantes utilizan las siguientes técnicas para mantener persistencia en un sistema:\n\n*   **Software Extensions (T1176)**: Abusan de las extensiones de software para establece
r acceso persistente. Estas extensiones son componentes modulares que mejoran o personalizan la funcionalidad de aplicaciones como navegadores web, IDEs y otras plataformas.\n*   **Browser Exten
sions (T1176.001)**: Una sub-técnica de Software Extensions, donde los atacantes abusan de las extensiones de navegadores web para obtener acceso persistente.\n*   **Create or Modify System Proc
ess (T1543)**: Crean o modifican procesos a nivel de sistema para ejecutar repetidamente cargas útiles maliciosas. Esto incluye la instalación de nuevos servicios, daemons o agentes, o la modifi
cación de los existentes.\n*   **Server Software Component (T1505)**: Abusan de las características de desarrollo extensibles legítimas de los servidores para establecer acceso persistente.\n*
**Compromise Host Software Binary (T1554)**: Modifican binarios de software del host para establecer acceso persistente. Esto puede implicar reemplazar o infectar un binario legítimo con una pue
rta trasera, o parchear funcionalidad maliciosa en un binario existente.",
  "sources": [
    {
      "id": "8566ea32-aefd-5474-80a2-9e9e484aa78e",
      "score": 0.7492026,
      "source": "mitre",
      "title": "Software Extensions",
      "text": "ATT&CK ID: T1176\nTechnique name: Software Extensions\nDescription: Adversaries may abuse software extensions to establish persistent access to victim systems. Software extensions
are modular components that enhance or customize the functionality of software applications, including web browsers, Integrated Development Environments (IDEs), and other platforms.(Citation: Ch
rome Extension C2 Malware)(Citation: Abramovsky VSCode Security) Extensions are typically installed via official marketplaces, app stores, or manually loaded by users, and they often inherit the
permissions and access levels of the host application. Malicious extensions can be introduced through various methods, including social engineering, compromised marketplaces, or direct installat
ion by users or by adversaries who have already gained access to a system. Malicious extensions can be named similarly or identically to benign extensions in marketplaces. Security mechanisms in
extension marketplaces may be insufficient to detect malicious components, allowing adversaries to bypass automated scanners or exploit trust established during the installation process. Adversa
ries may also abuse benign extensions to achieve their objectives, such as using legitimate functionality to tunnel data or bypass security controls. The modular nature of extensions and their i
ntegration with host applications make them an attractive target for adversaries seeking to exploit trusted software ecosystems. Detection can be challenging due to the inherent trust placed in
extensions during installation and their ability to blend into normal application workflows.\nTactics: Persistence\nTactic shortnames: persistence\nPlatforms: linux, macos, windows\nIs sub-techn
ique: False\nMitigations: User Training, Limit Software Installation, Execution Prevention, Audit, Update Software\nSource: MITRE ATT&CK Enterprise"
    },
    {
      "id": "de679203-e19c-5b9b-afba-1a430bc76ac2",
```


Download

Aunque en la imagen no se aprecian más contextos vemos en la respuesta del LLM como hace referencia a distintas técnicas como son la T1176, T1176.001, T1543, T1505 y T1554, lo que indica que ha usado los contextos recuperados de la base vectorial (recodar que el top_k está establecido en 5).

10.3 Pruebas de consultas sobre análisis de amenazas

Estas pruebas van enfocadas a la fuente ENISA. Como ejemplo se realiza la siguiente consulta:

> *"¿Cuáles son las principales amenazas de ciberseguridad en Europa?"*

La respuesta del sistema es la siguiente:

```
{
  "question": "¿Cuáles son las principales amenazas de ciberseguridad en Europa?",
  "answer": "Las principales amenazas de ciberseguridad en Europa, según la información proporcionada, son:\n\n*   **Actividades de ciberdelincuencia:** Representan el 81% de las actividades obs
ervadas.\n*   **Ransomware:** El sector manufacturero es el más afectado. Se ha observado una fragmentación en el ecosistema del ransomware, con la aparición de nuevas variantes y programas de R
ansomware-as-a-Service (RaaS). Akira, SafePay y Qilin son algunas de las variantes más desplegadas.\n*   **Info-stealers (ladrones de información):** Son un vector de amenaza constante, utilizad
os principalmente para el robo de credenciales, secuestro de sesiones y intermediación de acceso. Lumma info-stealer ha visto un aumento significativo en su uso.\n*   **Brechas de datos:** Afect
an principalmente a la administración pública, infraestructura y servicios digitales, y finanzas. Estos datos suelen ser vendidos por Initial Access Brokers (IAB).\n*   **Ingeniería social:** La
s tácticas de ingeniería social, especialmente el phishing (incluyendo vishing, malspam y malvertising), son el principal punto de entrada para los actores de amenazas, representando aproximadam
ente el 60% de los casos observados.\n*   **Explotación de vulnerabilidades:** Sigue siendo un vector de intrusión prevalente.\n*   **Botnets:** Representan una parte significativa de las amenaz
as.\n*   **Aplicaciones maliciosas:** El software y las aplicaciones comprometidas o troyanizadas continúan siendo un factor en las intrusiones.\n*   **Amenazas móviles:** Constituyen la mayor p
arte de las categorías de amenazas.\n*   **Amenazas web:** La explotación de servicios y aplicaciones en línea persiste.\n*   **Amenazas de tecnología operativa (OT):** Reflejan la creciente exp
osición de sistemas industriales y críticos.\n*   **Riesgos en la cadena de suministro:** Los atacantes aprovechan vías indirectas a través de terceros.\n*   **Amenazas asociadas a hacktivistas:
** Incluyen ataques de Denegación de Servicio Distribuido (DDoS), que persisten como una molestia.\n*   **Ciberespionaje:** Las actividades de ciberespionaje son estables, con una prevalencia co
ntinua de conjuntos de intrusión con nexos en Rusia y China.\n*   **Actividades de ciberdelincuentes oportunistas:** Dominan el panorama de amenazas, utilizando ransomware e info-stealers.\n*
**Uso de herramientas para evadir defensas:** Se observa un aumento en el uso de herramientas para desactivar EDR (Endpoint Detection and Response) y técnicas como BYOVD (Bring Your Own Vulnerab
le Driver).\n*   **Intensificación de amenazas:** Se espera que la actividad de amenazas se intensifique en convergencia, automatización e industrialización, con la IA acelerando la innovación o
fensiva.",
  "sources": [
    {
      "id": "f1767282-a1f4-5d2e-80e6-286fa4b238a6",
      "score": 0.72814476,
      "source": "enisa",
      "title": "KEY CYBERCRIME THREATS",
      "text": "riminal groups. \n6.1 KEY CYBERCRIME THREATS \nBased on monitored Data Leak Sites (DLS) and cybercriminal forums, cybercrime claims accounted for 81% \nof activities. \nKnown EU v
ictims include a broad range of sectors, with at \nleast 36 sectors identified in total, including critical sectors \nas shown in the NIS2 Directive, with DIS and the \nmanufacturing sector rema
ining the most impacted in the \nEU. \nOver the reporting period, data breaches primarily impacted \nEU digital infrastructure and services (27.7%), notably \nthrough the sale
rom telecommunications \nproviders, followed by the sale of data related to public \nadministration (17%). Ransomware claims were made \nprimarily against the manufacturing se
le the recorded share of ransomware deployments \nremained stable, a shift in the ransomware ecosystem \nwas observed over the reporting period, marked by a \ncontinuous fragmentation, ultimatel
```


En este caso vemos que la respuesta tiene relación semántica tanto con los contextos recuperados como con la pregunta realizada.

10.4 Preguntas fuera del contexto de las fuentes de datos

Estas pruebas van enfocadas a ver cómo responde el LLM y la consulta a la base vectorial cuando las preguntas no guardan relación con las fuentes de datos. Como ejemplo se realiza la siguiente consulta:

"¿Qué tiempo va a hacer mañana en Málaga?"

La respuesta del sistema es la siguiente:


```
{
  "question": "¿Que tiempo va a hacer mañana en Málaga?",
  "answer": "La información proporcionada no contiene detalles sobre el pronóstico del tiempo para mañana en Málaga.",
  "sources": [
    {
      "id": "a3156000-a078-54f8-850d-4f498e6c1f36",
      "score": 0.5468551,
      "source": "kev",
      "title": "Smartbedded Meteobridge Command Injection Vulnerability",
      "text": "CVE: CVE-2025-4008\nVendor: Smartbedded\nProduct: Meteobridge\nVulnerability name: Smartbedded Meteobridge Command Injection Vulnerability\nDescription: Smartbedded Meteobridge co
ntains a command injection vulnerability that could allow remote unauthenticated attackers to gain arbitrary command execution with elevated privileges (root) on affected devices.\nRequired acti
on: Apply mitigations per vendor instructions, follow applicable BOD 22-01 guidance for cloud services, or discontinue use of the product if mitigations are unavailable.\nDue date: 2025-10-23\nK
nown ransomware campaign use: Unknown\nCWES: cwe-306, cwe-77\nNotes: https://forum.meteohub.de/viewtopic.php?t=18687 ; https://nvd.nist.gov/vuln/detail/CVE-2025-4008\nSource: CISA Known Exploite
d Vulnerabilities Catalog"
    },
    {
      "id": "35b24dc0-18d0-55a6-85ca-de0060b33821",
      "score": 0.53769875,
      "source": "enisa",
      "title": "OUTLOOK & CONCLUSION",
      "text": "63 \n \n11. OUTLOOK & CONCLUSION \n \nIn the near-term, it is highly likely public and private organisations in EU MSs will continue to face hacktivist-\nassociated threats with p
eriodic peaks, stable cyberespionage activities with a continued prevalence of \nRussia-nexus and China-nexus intrusion sets, and an even more mature yet further fragmented cybercriminal \necosy
stem. \n \nIn terms of impact, the EU threat picture will remain dominated by opportunistic cybercriminal activities \ninvolving the use of ransomware and information-stealers, despite the achie
vements of law-enforcement. \nDisplaced or disrupted RaaS brands will continue being promptly replaced by emerging programmes. The \ncriminal marketplace will continue formalising around skills
to further scale campaigns, notably through AI \nintegration, IoT and large-scale exploitations of vulnerabilities and the targeting of critical sectors, notably \nhosting companies and IT provi
ders. The rising use of EDR-kill tooling (e.g., AvNeutralizer, EDRKillShifter) \nand BYOVD, as well as legal-pressure features in extortion playbooks, will sharpen both the sp
of intrusions. Hacktivist-led DDoS will persist as a nuisance, both in terms of the disruption of \nbusiness continuity and in the information operation sphere, highly likely
high visibility \nevents and announcements by EU MSs and EU entities and authorities. State-nexus intrusion sets will \ncontinue to blend espionage, supply-chain access and IO,
```


Podemos observar que la base de datos ha recuperado información de distintas fuentes como son ENISA y KEV pero el LLM no ha visto relación semántica entre la pregunta y los contextos por lo que responde con:

"La información proporcionada no contiene detalles sobre el pronóstico del tiempo para mañana en Málaga."

11 Limitaciones del sistema

11.1 Interpretación del contexto por parte del LLM

Como se ha visto en las pruebas, el LLM puede interpretar de forma incorrecta determinados campos de los textos recuperados.

11.2 Ausencia de eliminación automática de información obsoleta

Aunque el sistema permite realizar actualizaciones de forma incremental en la base de datos vectorial, no se ha implementado un sistema que permita eliminar automáticamente los datos o documentos que dejan de aparecer en las fuentes originales, provocando con el paso del tiempo que la base de datos mantenga información obsoleta.

12 Mejoras futuras

A partir de las limitaciones descritas anteriormente se pueden añadir mejoras en las siguientes versiones del sistema.

12.1 Optimización del prompt usado por el LLM

Para evitar que el LLM no sepa interpretar determinados campos de los contextos recuperados sería necesario proporcionar instrucciones más claras al modelo para que sepa cómo interpretar los campos.

12.2 Incorporación de un sistema de reranking

Actualmente el sistema recupera los chunks mas relevantes mediante una búsqueda vectorial por similitud. Incorporar un modelo de reranking que reordene los resultados recuperados antes de pasarlos al LLM podría mejorar el contexto utilizado para generar la respuesta.

12.3 Establecimiento de un umbral mínimo de similitud

Establecer un umbral mínimo de similitud para filtrar los resultados recuperados antes de pasarlos como contexto al LLM permitiría mejorar la calidad de las respuestas generadas por el sistema evitando así que se generen respuestas con fragmentos poco relevantes.